



Trabajo Práctico N°2

Análisis y Síntesis de Audio digital

22.05 - Análisis de Señales y Sistemas Digitales

Grupo N°1

Titular de cátedra: Ing. Jacoby, Daniel Andrés
Adjunto: Ing. Paura Bersán, Martín

Legajo N°	Alumnos
61427	Gonzalo Ezequiel Linares
61431	Nicolás Bustelo
61467	Damián Ezequiel Sergi
61885	Agustín Luis Gullino
62012	Santiago Feldman

17/05/2023

Índice

1. Programa Principal	2
2. Síntesis	2
2.1. Síntesis FM	2
2.2. Síntesis de sonidos mediante modelos físicos	4
2.3. Síntesis basada en muestras	6
2.3.1. <i>Time Shifting</i>	7
3. Efectos de Audio	9
3.1. Eco simple	9
3.2. Flanger	10
3.3. Reverberación de Schroeder	10
Referencias	12

1. Programa Principal

Se desarrollo un programa en *Python* capaz se sintetizar una pista a partir de un archivo MIDI. El programa es capaz de:

- Cargar un archivo en formato MIDI
- Asignar cualquier instrumento a cualquier track.
- Mezclar los distintos tracks para generar una pista.
- Agregar y configurar diferentes efectos, para cada track y para la mezcla total.
- Guardar la pista generada en algún formato de audio (wav, mp3, etc).
- Mostrar el espectrograma del audio generado (ver el punto correspondiente del TP).

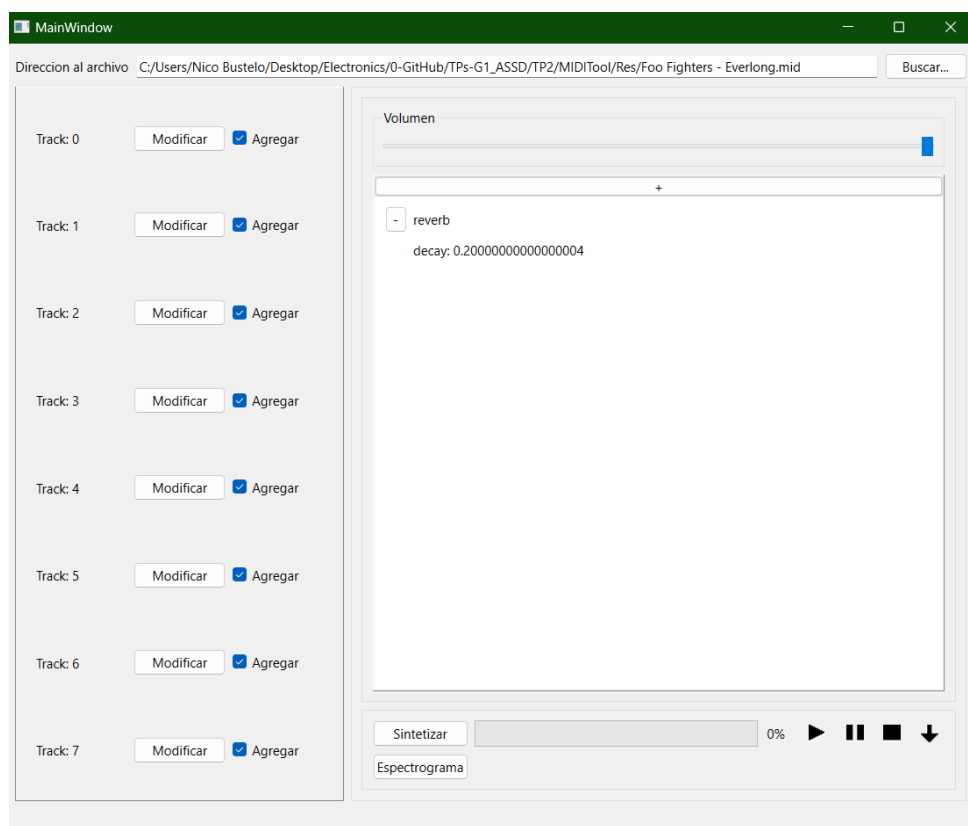


Figura 1: GUI del programa.

2. Síntesis

Para la generación del sonido de los diferentes instrumentos se utilizaron diversas técnicas de sintetización digital.

2.1. Síntesis FM

Para la síntesis FM de un clarinete se utilizó la ecuación:

$$x(t) = A(t) \cdot \cos(2 \cdot \pi \cdot f_c \cdot t + I(t) \cdot \cos(2 \cdot \pi \cdot f_m \cdot t + \Phi_m) + \Phi_c) \quad (1)$$

Inicialmente sabemos que $\Phi_m = \Phi_c = -\pi/2$. Luego, gracias a la bibliografía [1] podemos deducir que la frecuencia central de cada nota musical es el mínimo común múltiplo entre la f_m y f_c . En dicho documento, indica que para poder realizar un sonido similar al del clarinete es necesario que f_m sea dos veces y que f_c sea tres veces la frecuencia central de la nota. En lo que respecta al índice de modulación, si bien depende del tiempo, es constante en cierto punto y su valor está determinado por el cociente f_c/f_m . La curva inicial se aproximó a una función cuadrática que arrancaba en cero y en 0.1 segundos su valor era el nominal correspondiente.

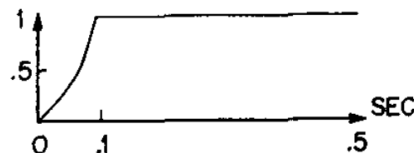


Figura 2: Envolvente de la función índice de modulación para obtener un sonido similar a un instrumento de viento.

En lo que respecta a la amplitud de salida se definió en función del volumen máximo que se necesita en el programa debido a que al tener una función acotada como el coseno, $A(t)$ define la amplitud máxima de la señal de salida. Dicha función en el tiempo se puede ver en la [Figura 2](#) donde la parte inicial se calcula de manera idéntica al del índice de modulación y en la curva final se implementó una función lineal para simplificación.

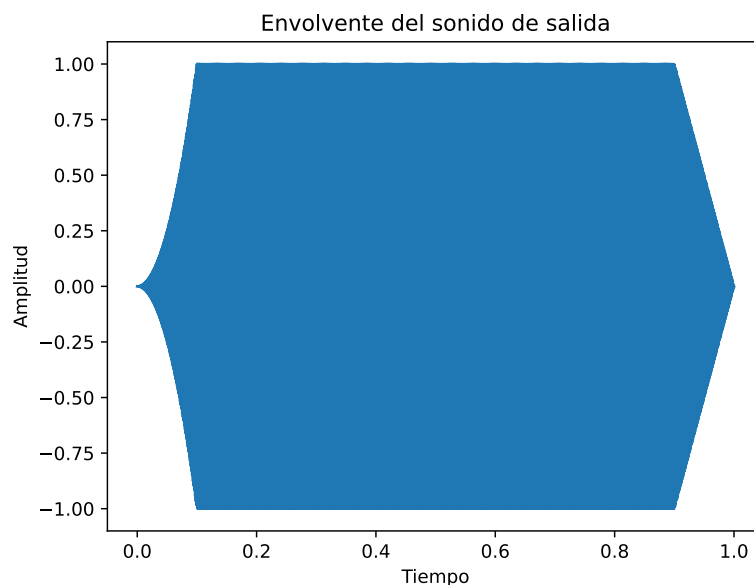


Figura 3: Envolvente de la amplitud de sonido en función del tiempo para una muestra de un tono de 1 segundo de duración.

Para poder obtener otro instrumento de viento es necesario modificar f_c y f_m para lograr otros armónicos. Por ejemplo para generar un sonido similar al de un fagot, $f_c = 5.f_o$ y $f_m = f_o$ teniendo un índice de modulación igual a 5. En otro caso si se quiere realizar sonidos de percusión, aparte de modificar el índice de modulación es necesario modificar la envolvente de la amplitud y tener una función exponencial decreciente por ejemplo.

El uso de la síntesis por FM proporciona un control muy simple del espectro y una manera muy sencilla de general ciertos instrumentos.

2.2. Síntesis de sonidos mediante modelos físicos

Para la implementación del sonido de una guitarra se utilizó el sistema de la [Figura 4](#) basado en el modelo de Karplus-Strong. La salida del sistema puede expresarse como:

$$y(n) = \frac{x(n) + x(n-1) + y(n-L) + y(n-(L+1))}{2}$$

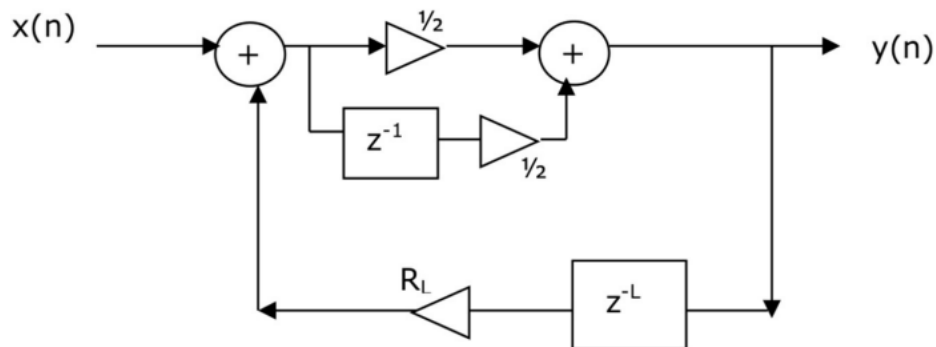


Figura 4: Sistema a implementar para la síntesis de sonido mediante modelo físico basado en el modelo de Karplus-Strong

Para su implementación en código, se resolvió el sistema y se obtuvo la $H(z)$ a implementar:

$$H(z) = \frac{Y(z)}{X(z)} = \frac{1 + z^{-1}}{2 - R_L z^{-L} - R_L z^{-(L+1)}}$$

Si graficamos la constelación de polos y ceros para $L = 40$ y $R_L = 0.99$, configuración la cual simularía el sonido de una guitarra eléctrica, podemos ver que obtenemos $L+1$ ceros y $L+1$ polos como muestra la [Figura 5](#). Variando los parámetros anteriores, podemos obtener transferencias muy similares entre ellas, con polos mas cercanos al centro de la circunferencia unitaria. Cabe destacar que los límites de R_L son entre $(0,1]$. Valores superiores a 1 harían que el sistema sea inestable.

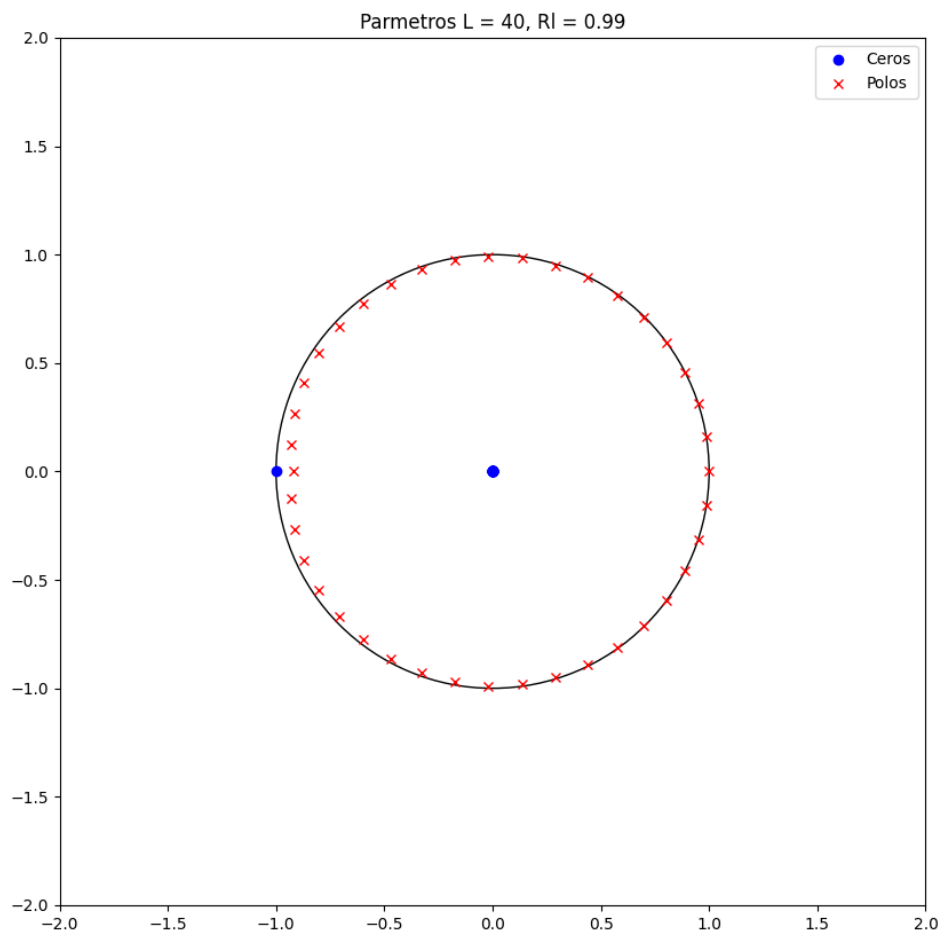


Figura 5: Diagrama de polos y ceros de una transferencia posible utilizando $L = 40$ y $R_L = 0.99$.

Si simulamos la salida para dos ruidos diferentes a la entrada del sistema, podemos obtener el las imágenes de la [Figura 6](#) y [Figura 7](#). Es notable que para el mismo tiempo de nota, la amplitud del sonido con entrada normal decae considerablemente mas rápido que la amplitud del sonido con la entrada uniforme. Esto se puede atribuir que dado que el ruido uniforme esta formado por números pseudoaleatorios, existe una mayor diferencia entre el valor actual de la señal y su anterior comparado al ruido uniforme. Esto ultimo genera mayores armónicos de alta frecuencia que se atenúan rápidamente al pasar por el sistema que se comporta como pasa-bajos en esencia.

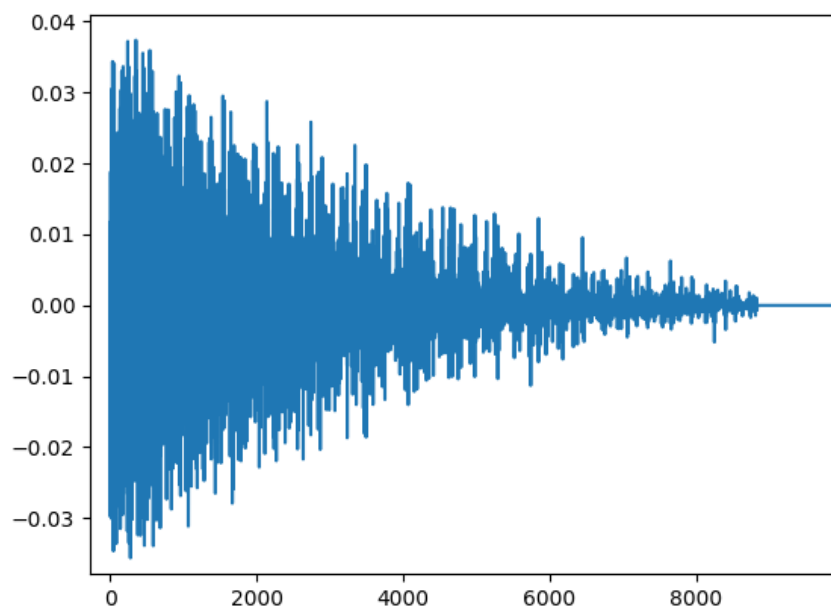


Figura 6: Señal de salida con ruido normal a la entrada

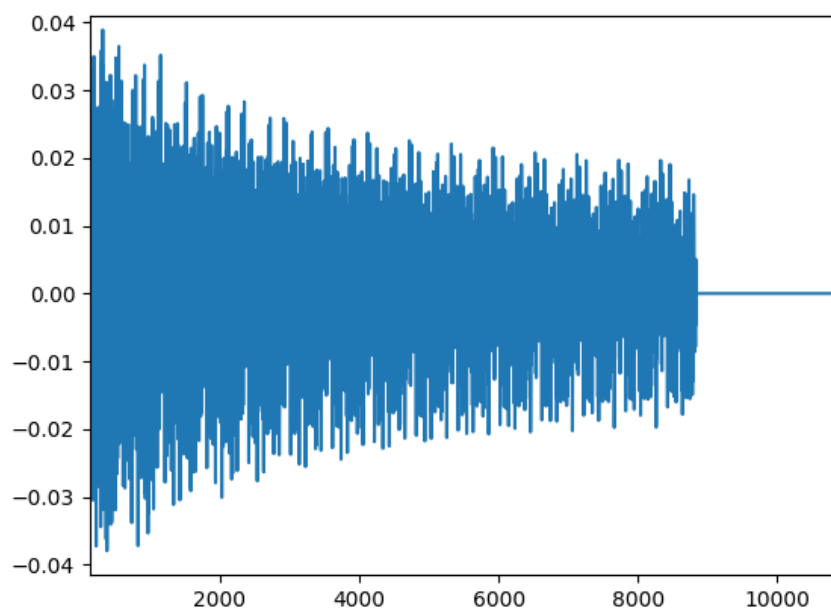


Figura 7: Señal de salida con ruido uniforme a la entrada

2.3. Síntesis basada en muestras

Para esta sección se utilizó la biblioteca *Librosa* y sus funciones para corrimiento de tiempo y corrimiento de tono. A continuación se explica el funcionamiento de ambas funciones para una comprensión completa del sintetizador.

2.3.1. Time Shifting

En este caso, *Librosa* utiliza un *Phase Vocoder*, cuya función se basa en el estiramiento del espectro a lo largo del tiempo. Internamente, el vocoder se implementa utilizando la *STFT*. En pocas palabras se toma cada bin y se realiza un *Upsampling*, o *Downsampling* correspondiente en el espectro, de forma que se consigue el mismo espectro (dada la correcta interpolación en cada bin) pero con una escala de tiempo mas estirada o acotada. El problema que aparece es la corrección en la fase. El algoritmo de *Librosa* realiza una interpolación lineal para el modulo del espectro, pero no se puede realizar lo mismo con la fase. Es mas, la fase tiene problemas con su continuidad, ya que esta restringida entre $-\pi$ y π , por lo que no se puede interpolar. Si se puede trabajar sobre la derivada de esta. Conociendo la cantidad de *Overlapping* entre la ventana de la *STFT* se puede calcular el avance de fase para cada ventana si se asume un coseno centrado en el centro del bin. Con esto se consigue un incremento de fase que se corresponde para cada frecuencia. Esto introduce error, ya que si la frecuencia no esta en el centro del bin se genera una distorsión de la fase alterando el sonido final de salida.¹²

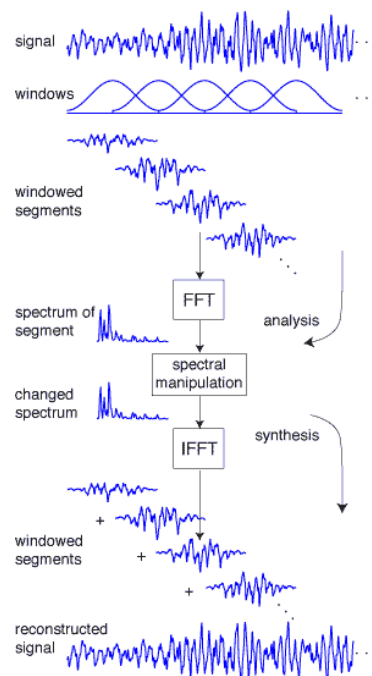


Figura 8: Diagrama básico del *Phase Vocoder*.

En la [Figura 8](#) se puede apreciar un diagrama simplificado del *phase Vocoder*. Este elemento tiene otro uso además del *Time Stretching*, también se puede utilizar en *Pitch Shifting*. Es sencillo demostrar que si una señal muestreada a 100kHz se asume muestreada a 50kHz (asumimos $f_s = 50\text{kHz}$ a pesar de ser 100kHz) el espectro original de la señal muestreada se vera comprimido a la salida de un parlante (asumiendo que la entrada era un sonido audible). Esa compresión en frecuencia se puede cuantificar como el cociente entre la frecuencia de muestreo original y la que se toma. Es importante observar que esto genera cambios en la duración de la señal, ya que tenemos la misma cantidad de puntos para una frecuencia de muestreo menor. Sabiendo esto, y utilizando *Time shifting* se puede obtener una señal estirada en tiempo que al disminuir su frecuencia de muestreo, vuelva a tener la misma duración temporal que a la entrada, manteniendo las relaciones en el espectro casi sin modificaciones. Entonces se mantienen las relaciones armónicas, pero las distorsiones aparecen al momento de querer remuestrear la señal para poder obtenerla en un formato

¹Sethares, William. .^A Phase Vocoder in Matlab". sethares.engr.wisc.edu.

²D. P. W. Ellis. .^A Phase Vocoder in Matlab". ee.columbia.edu/ dpwe/resources/matlab/pvoc/

genérico de $f_s = 44100\text{Hz}$. Este remuestreo se realiza de forma que introduzca el menor aliasing posible.

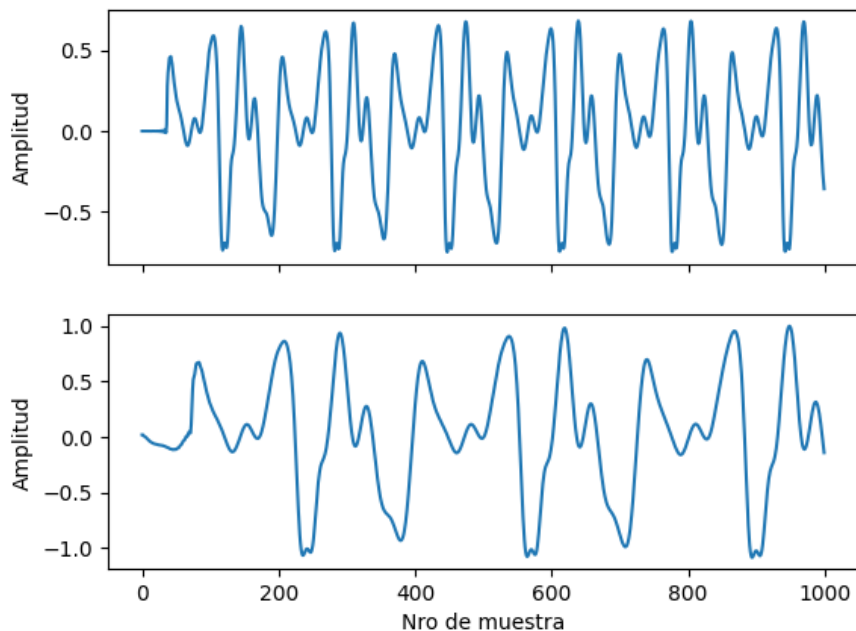


Figura 9: Extracto de las primeras 1000 muestras de un sonido proveniente del grito de un miembro del grupo de trabajo. El gráfico superior es la señal original en tono de C4, y el gráfico inferior es esa misma muestra corrida una octava mas abajo (C3).

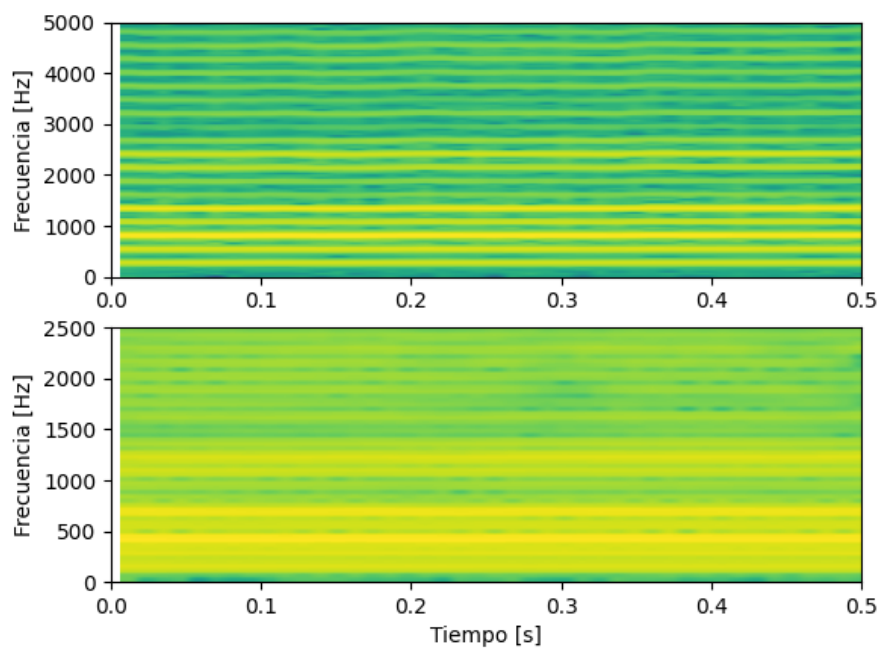


Figura 10: Diagrama espectral de las señales descritas en la [Figura 9](#).

La [Figura 10](#) y la [Figura 9](#) muestran un ejemplo del resultado de corrimiento de tono. Se puede ver que el resultado da señales bastantes similares en forma pero con un estiramiento en tiempo. También en el espectro se puede ver la contracción de las bandas espectrales concentrando la energía en frecuencias menores.

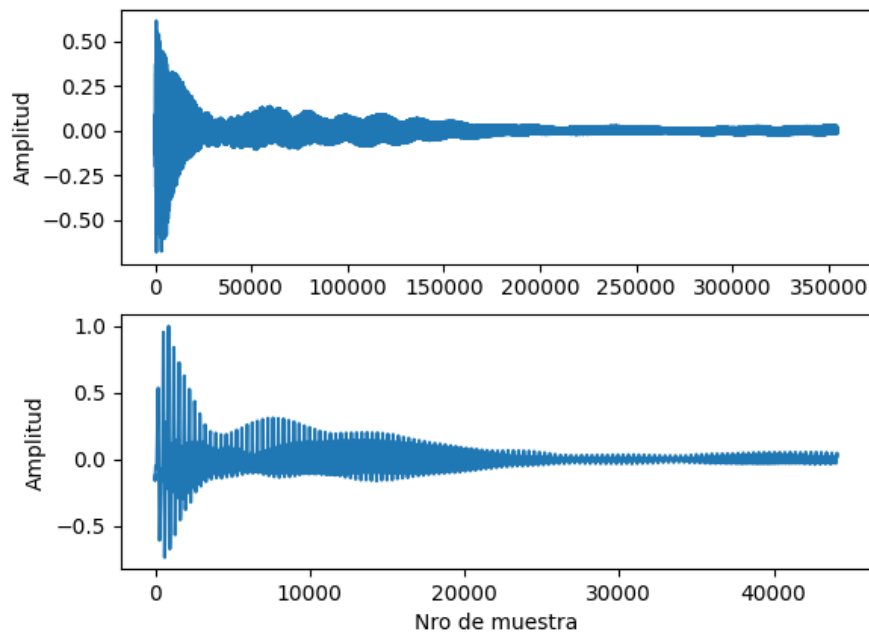


Figura 11: Demostración del estiramiento en tiempo y cambio de tono de una muestra de una nota C4 de Piano. El gráfico superior es la nota original, y el inferior el resultado.

Por ultimo se tiene la [Figura 11](#) donde se demuestran algunas de las imperfecciones de este método, donde se aprecia que la forma original de la señal es respetada, pero las respuestas transitorias suelen ser deformadas.

3. Efectos de Audio

Se implementaron 3 tipos de efectos de audio en el programa: *Eco simple*, *Flanger* y *Reverberación de Schroeder*. Los mismos se detallarán a continuación

3.1. Eco simple

El efecto de eco simple consiste en aplicarle un retardo fijo a la señal que se ingresa, atenuarla y luego mezclarla con la señal original. De esa manera, se solapará sobre la señal $x(n)$, muestras anteriores de la señal, generando así el efecto de eco. El efecto se puede diagramar mediante el siguiente sistema [2]:

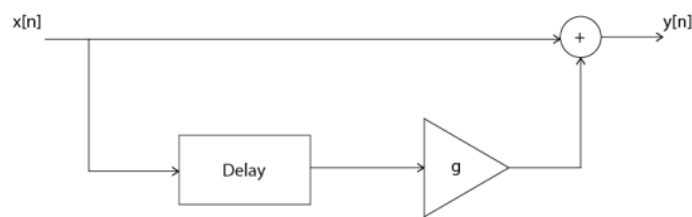


Figura 12: Implementación del sistema de Eco simple

Del diagrama en bloques del eco simple, se tiene que la salida del sistema es $y(n) = x(n) + g \cdot x(n - M)$, siendo g la atenuación del eco y M el número de retardos del eco (es decir, el tamaño de la línea de retardo; cuántos retardos unitarios se realizan sobre la señal original). Tanto M como g son parámetros que pueden ser ajustados por el usuario. Los valores preestablecidos son $g = 0,8$ y $M = 300$. g puede variar de 0 a 1 y se recomienda que el número de retardos unitarios esté entre 200 y 400 para lograr el efecto deseado.

3.2. Flanger

El efecto *flanger* consiste en mezclar la señal que se ingresa con la misma señal retardada en el tiempo. Dicho retardo no es fijo para cada muestra, sino que varía de acuerdo a un oscilador de baja frecuencia (LFO). El sistema correspondiente al efecto es el que se muestra en la Figura 12, con la diferencia de que la cantidad de retardos de la línea de retardo (el bloque *Delay*) es variable y varía de acuerdo a la frecuencia del LFO.

La salida del sistema es $y(n) = x(n) + g \cdot x(n - Mn)$. El parámetro g se denomina *depth* (profundidad) y determina qué tan acentuado es el efecto; va de -1 a 1 , siendo los valores negativos una inversión de fase de los valores positivos ($g = 0$ implica que el efecto es nulo). La cantidad de retardos en función de la frecuencia del LFO está dada por $M_n = M_o \cdot [0,5 + 0,5 \cdot \sin(2\pi n f)]$, siendo M_o un valor fijo para la tasa de muestreo del archivo que se carga. f se suele denominar *speed* (rapidez). Tanto g como f son parámetros variables por el usuario.

3.3. Reverberación de Schroeder

El algoritmo de reverberación de Schroeder [3] es un método para generar reverberación artificial utilizando una combinación de filtros *comb* centrados en distintas frecuencias y una cascada de filtros pasatodo, tal como lo indica el sistema siguiente:

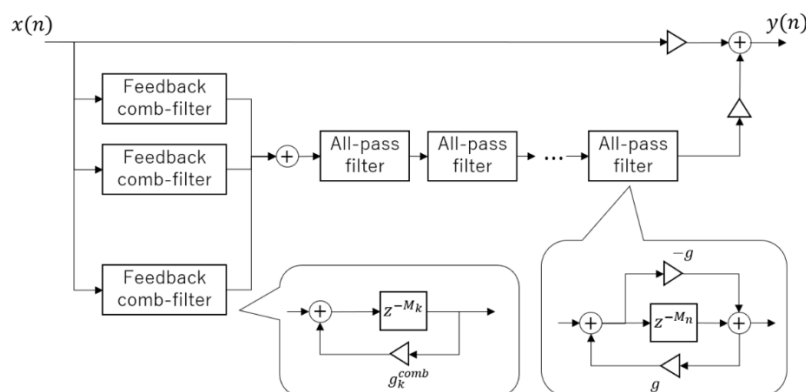


Figura 13: Implementación del sistema de reverberación de Schroeder

El código que realiza el sistema fue tomado de la siguiente página web [4], donde se dejaron los

coeficientes tal cual se encontraban ya que producían un efecto muy bien logrado. Los parámetros que se pueden configurar en la GUI son las ganancias de los amplificadores previos a la suma que se realiza en el sistema. Los mismos se nombraron como *Direct* a la ganancia que corresponde a $X(n)$ y *Reverb* a la correspondiente a los filtros.

Referencias

- [1] J. M. CHOWNING, *The Synthesis of Complex Audio Spectra by Means of Frequency Modulation*. dirección: <https://web.eecs.umich.edu/~fessler/course/100/misc/chowning-73-tso.pdf>.
- [2] A. R. V. .-. J. C. B. García, *Implementation of sound effects in DSP*.
- [3] M. R. Schroeder, *Natural Sounding Artificial Reverberation*, 1962.
- [4] Anónimo, *Schroeder Reverberator*. dirección: <https://www.wizard-notes.com/entry/asp/schroeder-reverberator>.